

# VCF-RDFizer: From a VCF File to Explicit, Verifiable, and Governable Semantic Genomic Data

Elias Crum<sup>ORCID</sup><sup>1,2\*</sup>, Bart Buelens<sup>ORCID</sup><sup>1</sup>,  
Ruben Taelman<sup>ORCID</sup><sup>2</sup>, Gökhan Ertaylan<sup>ORCID</sup><sup>1</sup>

<sup>2</sup>IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium.

<sup>1</sup>Digital Biological Systems, Flemish Institute for Technological Research (VITO), Boeretang 200, 2400, Belgium.

\*Corresponding author(s). E-mail(s): [elias.crum@ugent.be](mailto:elias.crum@ugent.be);

## Abstract

**Background:** The Variant Call Format (VCF) is widely used, compact, and readily parsable, but integrating its observations with changing biomedical evidence, clinical context, and data-use policies requires relationships beyond the file itself. Making these relationships explicit could support variant reanalysis, pharmacogenomic decision support, and governed data reuse, provided that the converted information remains checkable against its source. Here we present VCF-RDFizer, a command-line tool that converts VCF into the Resource Description Framework (RDF) using the public VCF Core vocabulary, as a first step toward these applications.

**Results:** We evaluate VCF-RDFizer using public genomic datasets, source-based validation, scaling experiments, and a linked workflow combining genomic observations with external knowledge and simulated access policies. The results show that VCF data can be converted into an explicit RDF representation while retaining the information needed to check the transformed data against its source. Complementary query- and constraint-based validation expose different classes of transformation errors, demonstrating the value of validating both data content and graph structure. The resulting RDF representations also support queries that combine genomic observations with external variant knowledge, population information, and requester-specific policies while reproducing the results of an equivalent conventional VCF workflow. These capabilities come with additional

conversion and indexing costs: selected repeated queries benefited from an existing graph index, whereas native VCF processing remained advantageous for the tested one-pass analyses and narrow indexed lookups.

**Conclusions:** VCF-RDFizer supports verifiable conversion and reusable, policy-aware research queries; it is not a universal replacement for VCF. Its value depends on the relationships needed, the validation coverage achievable, and the cost of conversion and indexing.

**Keywords:** Genomic variation, Variant Call Format, RDF, SPARQL, Semantic validation, Data-use policies

## 1 Background

The Variant Call Format (VCF) is widely used to represent and exchange genomic variation in research and clinical sequencing, including in pharmacogenomics and precision medicine [1–3]. It is compact, readily parsed, and already supports identifiers, reference information, and extensible annotations [1, 4]. However, many questions in healthcare and biomedical research require more than the variants recorded in a file. Practical uses combine genomic observations with clinical records, external biomedical knowledge, sample metadata, and conditions governing data use [5–7]. The challenge is therefore not simply to read VCF, but to make the relationships needed for downstream uses explicit and reusable across independently maintained resources. Three generalized use cases illustrate such needs.

### ***\*\*(1) Clinical variant interpretation and reanalysis.\*\****

Clinical interpretation combines genomic observations with clinical assertions, population frequencies, gene–disease associations, and patient phenotypes, all of which can change over time [8, 9]. Reanalysis can therefore identify new pathogenic or likely pathogenic findings and downgrade earlier classifications [10]. The observation and the evidence used to interpret it have different lifecycles. Keeping the relationship explicit and reusable between observation and the evidence is difficult when this interpretation is stored only in annotations or application-specific databases.

### ***\*\*(2) Pharmacogenomic clinical decision support.\*\****

Pharmacogenomic decision support connects genomic observations to an interpreted phenotype, a medication, an indication, and clinical guidance. For example, CYP2C19-guided clopidogrel recommendations depend on both metabolizer phenotype and clinical indication [11]. Current systems still face challenges in representing genomic results as discrete data, integrating them with electronic health records (EHRs), and maintaining the knowledge used for interpretation [12, 13]. These relationships span systems and must remain traceable if genomic observations are to support changing clinical decisions.

***\*\* (3) Consent-aware genomic data reuse. \*\****

Secondary use adds a governance requirement to these integration problems. Data access decisions must reflect the uses permitted by participant consent, which can differ across participants, datasets, and purposes [7, 14]. The GA4GH Data Use Ontology (DUO) and Machine Readable Consent Guidance provide machine-readable terms for expressing these conditions [7, 14, 15]. Because genomic data are highly sensitive [16, 17], selective release of participants, regions, or variants must remain tied to the conditions that govern their use.

None of these tasks is impossible with the current VCF representation. Variant-analysis frameworks, clinical systems, and data-access infrastructures already address important parts of these workflows [7, 12, 18]. The remaining challenges increasingly concern how these systems interoperate, how computational and operational costs are balanced, and how analyses can be reproduced as external knowledge and policies change. VCF itself is not designed to provide a shared representation linking a source observation to evolving evidence, patient context, and the conditions governing a particular request. These relationships are therefore commonly reconstructed in annotations, local databases, or application logic surrounding the file(s). As a result, there have become many disparate, narrowly scoped solutions for connecting variant data to external data and, thus, motivates a shared representation in which these relationships can be expressed explicitly, reused across applications, and queried independently of the source file(s).

The Resource Description Framework (RDF) provides one way to realize such a representation. RDF expresses relationships as triples whose resources can be identified by Internationalized Resource Identifiers (IRIs) [19, 20]. Linked Data practices use shared identifiers to connect independently managed resources, while SPARQL provides a language for querying those relationships [21–25]. For genomic variants, NCBI’s Sequence Position Deletion Insertion (SPDI) data model provides such a shared identity, so that the same allele reported in different files or in resources such as dbSNP and ClinVar can be recognized as one [26]. This allows an observation in a source file to remain distinct from the variant it describes, the evidence used to interpret it, and the policies governing its use. New evidence, prescribing guidance, or data-use conditions can therefore be represented as separate statements and queried alongside the observation without rewriting the source file or re-encoding the relationship for each application. The Open Digital Rights Language (ODRL) can similarly express policies over identified resources [27]. Realizing these benefits still depends on well-defined variant identities, consistent interpretation rules, and policy-evaluation mechanisms that act on the represented data [27, 28]. RDF therefore provides the representational foundation for these workflows, while normalization, clinical interpretation, and access enforcement remain responsibilities of the systems built around it.

The potential of RDF for VCF representation has already motivated several semantic-genomics initiatives and VCF-to-RDF converters (Table 1). Many of these tools were developed for focused applications or as components of larger pipelines, and consequently represent only the VCF information required for those tasks [29–33]. Existing tools also differ in their RDF models, treatment of header and genotype information, and mechanisms for extending the resulting representation [30, 32, 34–36].

The limited preservation of source context and single predefined application scoping of existing VCF-to-RDF approaches leaves an open need for a reproducible conversion that supports more generalized linking, validation, and downstream reuse. Non-RDF frameworks such as Hail, GEMINI, and TileDB-VCF, and standards such as Beacon, htsget, and FHIR Genomics, address complementary parts of variant analysis, retrieval, and clinical exchange but do not directly address the challenge of connecting VCF data to external data reliably [18, 37–41]. The goal of an RDF representation is therefore not to replace these systems, but to provide a reusable semantic layer through which genomic observations can be connected to them and to other external resources.

**Table 1** Documented capabilities of VCF-to-RDF converters and related workflows, from public documentation inspected in September and October 2026, not measurements on a shared input.

| Tool or workflow                              | Representation |                   |                   | Operation |                       |                |                |
|-----------------------------------------------|----------------|-------------------|-------------------|-----------|-----------------------|----------------|----------------|
|                                               | RDF model      | Header + genotype | Editable mappings | CLI       | Reproducible workflow | Compressed RDF | Public release |
| <i>Dedicated VCF-to-RDF converters</i>        |                |                   |                   |           |                       |                |                |
| <b>VCF-RDFizer</b> (this study)               | ●              | ●                 | ●                 | ●         | ●                     | ●              | ●              |
| TogoVar VCF2RDF [32]                          | ●              | ○                 | ●                 | ●         | ●                     | ○              | ●              |
| JVarkit VcfToRdf [30]                         | ●              | ●                 | ○                 | ●         | ●                     | ○              | ●              |
| SPARQLing Genomics [34, 35]                   | ●              | ●                 | ○                 | ●         | ●                     | ●              | ●              |
| VCF2RDF (web) [42]                            | ●              | ●                 | ○                 | ●         | ●                     | ○              | ●              |
| BioHackathon vcf2rdf [29, 43]                 | ●              | ●                 | ○                 | ●         | ●                     | ○              | ●              |
| <i>General genomic-format tools</i>           |                |                   |                   |           |                       |                |                |
| BioInterchange [36, 44]                       | ●              | ●                 | ●                 | ●         | ●                     | ○              | ●              |
| bio-vcf [31]                                  | ○              | ●                 | ●                 | ●         | ●                     | ○              | ●              |
| ruby-rdf/rdf-vcf [45]                         | ○              | ○                 | ○                 | ●         | ●                     | ○              | ●              |
| <i>Knowledge-graph construction pipelines</i> |                |                   |                   |           |                       |                |                |
| VariantKG [46]                                | ●              | ●                 | ○                 | ●         | ●                     | ○              | ●              |
| AgroLD VCF2RDF [47]                           | ●              | ○                 | ○                 | ○         | ○                     | ○              | ●              |
| <i>Virtual semantic access</i>                |                |                   |                   |           |                       |                |                |
| Semantic Beacons [33]                         | ●              | ○                 | ●                 | ○         | ●                     | ○              | ●              |

● supported and documented   ● partial or limited   ○ not found in public documentation. Supplementary Table S1 defines each column and gives the evidence for every mark.

For these applications, producing RDF is only a first step. Its usefulness also depends on whether the representation can be checked against the source and whether the costs of producing and using it are practical. Biomedical semantic tools such as SPHN Schema Forge, FAIR-Checker, and a FHIR RDF enrichment pipeline provide useful precedents for evaluating transformations alongside the validation and information requirements of their intended workflows [48–50].

We present VCF-RDFizer, a tool that converts VCF into RDF using the public VCF Core vocabulary and evaluates the resulting representation [51]. The vocabulary was developed separately; here, we focus on the representation choices needed to assess the converter and its downstream use [52]. We evaluate three capabilities together: source-verifiable conversion, linking to external knowledge maintained separately from the genomic data, and policy-aware release. A linked research workflow combines genomic observations, ClinVar classifications [53], population frequencies, and simulated data-use policies, with an equivalent conventional VCF workflow as its comparator. The experiment captures shared requirements of the three motivating use cases without claiming to implement clinical reanalysis, pharmacogenomic decision support, or a secure data-access service. Three questions organize the evaluation: **RQ1**: Which aspects of VCF information are preserved and verifiable? **RQ2**: What do the explicit VCF field representations enable in a linked, policy-aware workflow? **RQ3**: Under which workloads and representation choices are the costs of conversion, evaluation, linking, and querying practical? Together, these experiments evaluate whether VCF can serve as the basis for a reusable semantic representation that remains verifiable against its source, can be connected to external information, and provides a practical foundation for the broader use of VCF data.

## 2 Implementation

VCF-RDFizer<sup>1</sup> is a command-line tool that converts VCF files into RDF and validates the result against its source (Figure 1). The tool is available through PyPI [54], conda-forge [55], and Docker Hub [56], and runs its conversion toolchain in a single pinned Docker image, so that results do not depend on locally installed versions. The tool offers various functionalities including conversion, compression, decompression, validation, data linking, and policy attachment with documentation, usage guides, and examples explained extensively in the tool’s [README.md](#).

### 2.1 Representation of VCF

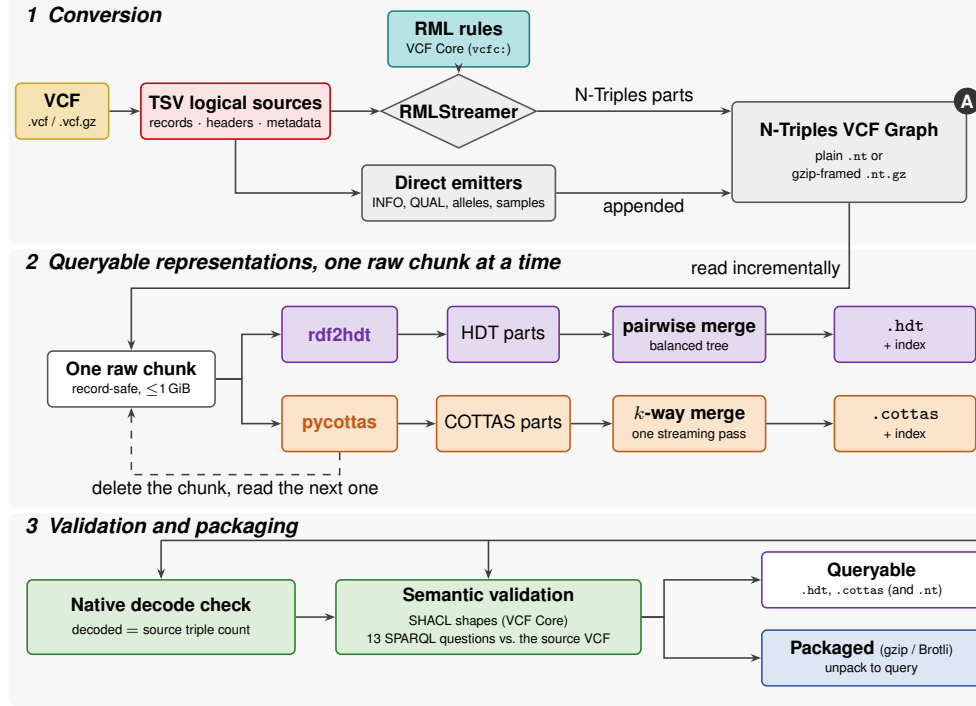
VCF-RDFizer generates a knowledge graph that represents the source VCF document, not only an interpreted variant. The conversion is built using the VCF Core Vocabulary [51, 52]<sup>2</sup> whose RDF output separates file and header assertions, variant records, and sample/genotype assertions (Figure 2). Supplementary Figure S1 gives the complete model, and the vocabulary itself is described separately [52]. The vocabulary aligns with FALDO for coordinates, the Sequence Ontology for variant types, HERO for genomic constructs, and features of the GA4GH Variation Representation Specification (VRS) [57–61]. These alignments connect VCF-specific structure to existing models; they do not replace the need to preserve the source’s interpretation context.

Deterministic IRI templates identify resources by source file and record position. The same biological alteration in two files therefore remains two observations, each with its own quality values, annotations, and provenance. A separate linkset can connect both to a shared variant identifier (Supplementary Figure S2). File-scoped

---

<sup>1</sup>Source code and implementation repository: <https://github.com/ecrum19/VCF-RDFizer>.

<sup>2</sup>Vocabulary documentation: <https://ecrum19.github.io/vcf-core-vocabulary/>.



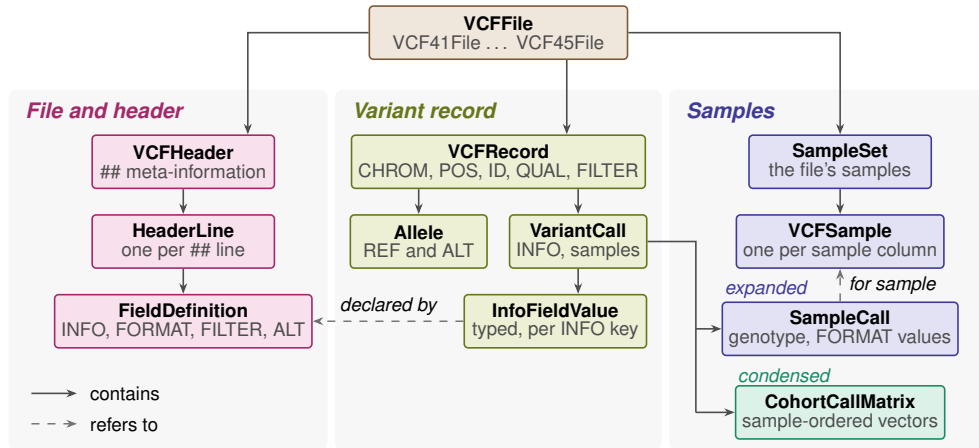
**Fig. 1 VCF-RDFizer v3.1.0 implementation.** RML mappings and direct emitters produce the source graph, which is appended incrementally in plain or gzip-framed N-Triples. Optional HDT and COTTAS construction uses one raw chunk at a time, followed by merging and indexing. Decoding/count checks against the N-Triples VCF graph (A), source comparisons, and enabled shape profiles assess different properties. Gzip and Brotli packages serve transport or archival use; their compression is distinct from a queryable format.

observation identity and cross-file variant identity serve different purposes and are not collapsed.

Two sample profiles offer different access granularity. *Expanded* makes sample calls and their FORMAT values directly addressable resources, whereas *condensed* retains them in ordered vectors that require decoding. Both preserve file, header, and record context. The different sample representation profiles primarily impact the conversion of single-sample VCF files versus cohort-scale multi-sample VCF files. The choice affects how individual sample values can be queried, not only the size and number of triples in the resulting RDF representation. Supplementary Section S2 describes the representation model and scaling derivations in greater detail.

## 2.2 Conversion and validation

A conversion run first rewrites each VCF as tab-separated tables, then maps them to RDF and, optionally, builds compressed representations and validates the RDF result. RDF Mapping Language (RML) mappings executed by RMLStreamer produce the record backbone, while direct emitters supply the remaining typed values,



**Fig. 2 Central terms of the VCF Core Vocabulary model emitted by VCF-RDfizer.** A versioned file contains its header, records, and sample set; a record contains its alleles and its call, which holds INFO values and, per sample, either sample calls (expanded profile) or sample-ordered vectors (condensed profile). Values refer back to the header declarations that define them. All terms use the `vcfc:` prefix; Supplementary Figure S1 adds subclasses, value items, genotypes, and property names.

header attributes, and sample structures [62, 63]. The implementation is therefore hybrid: changing the mappings alone does not change the complete representation. Output is appended incrementally to plain or gzip-framed N-Triples. HDT, COTTAS, and additional gzip or Brotli packaging of RDF outputs are optional artifacts, not prerequisites for obtaining or querying RDF [64–66]. Native decoding validation compares the triple count of each HDT or COTTAS output with that of the source graph, the N-Triples VCF graph produced by conversion (A in Figure 1). Supplementary Section S2.2 describes the emitters, chunk construction, and query paths in more detail, including the distinction between native compressed access and QLever’s separate index [67–69].

Validation, run after conversion or on existing RDF, compares thirteen SPARQL results with corresponding answers derived independently from the source VCF using `cyvcf2` and, for exact `FILTER` text, `bcftools` [4, 70, 71]. The comparisons cover record and genotype summaries, metadata and graph inventories, and summaries of values associated with their source records. After normalization, corresponding results are compared for exact agreement, although the value summaries do not constitute exhaustive value-by-value comparisons. Preflight queries and Shapes Constraint Language (SHACL) constraints assess additional structural properties of the graph [72]. The default shapes check cardinality and datatypes, while deeper profiles assess uniqueness and value agreement. Table 2 summarizes the validation coverage for each part of the representation and its limits. The complete query definitions, normalization rules, shape profiles, and execution limits are provided in Supplementary Section S3.

**Table 2** Representation contract and assessment boundary. Supplementary Table S2 gives the complete capability matrix; Q1–Q13 are the source-comparison queries (Supplementary Table S3) and shapes the Shapes Constraint Language (SHACL) constraints (Section 2.2).

| Information        | Representation                                                    | Checked by                 | Limit                                         |
|--------------------|-------------------------------------------------------------------|----------------------------|-----------------------------------------------|
| File and header    | Versioned file, declarations, and linked metadata                 | Q7–Q10, shapes             | Attribute–value agreement needs deeper shapes |
| Record and alleles | Core fields, ordered alleles, typed and explicitly missing values | Q1–Q4, Q11, shapes         | A summary digest, not exhaustive equality     |
| INFO               | Raw string plus optional typed, structured values                 | Q12, structural validation | Preserved, not interpreted                    |
| Samples and FORMAT | Expanded sample calls or condensed vectors                        | Q5, Q6, Q13                | Condensed values need decoding                |
| VCF versions       | 4.1–4.5 file classes; newer and symbolic fields retained          | Fixtures                   | Partly interpreted; local alleles unresolved  |
| Links and policies | Separate linksets; policy-defined release views                   | Use case                   | Identity assumptions and simulated policies   |

## 2.3 Linking and policy extensions

Two plug-ins extend the VCF RDF graph without modifying the converted source representation. `vcf-rdfizer-link` executes linkers declared in Turtle, with each linker defining a join strategy, reference resource, and emitted relationship. The plug-in writes the resulting links to a separate linkset and reports coverage and skipped records. In the evaluated workflow, linkers connect eligible alleles to SPDI identifiers and variant calls to overlapping genes. The SPDI linker harmonizes contig names and minimally trims explicit alleles, supporting consistent joins under the tested normalization assumptions without claiming canonical identity across all equivalent variant representations [26].

`vcf-rdfizer-policy` evaluates ODRL rules over files or declared graph selections, including genomic regions, named variants, and gene-linked records [14, 27]. The implemented policy profile applies default-deny and deny-overrides semantics, with permitted purposes drawn from a subset of DUO. Ownership rules define which dependent resources are removed when a file or record is withheld. For each requester, the plug-in materializes a release view, records which data were released or withheld, and validates the view against both the policy and an oracle derived from the source VCF text. This evaluation demonstrates policy-aware release under simulated rules, while anonymization, obligation enforcement, and deployment-level security remain outside the scope of the study. Supplementary Section S4 describes the linking tiers, policy evaluation, and release-validation procedure in detail.

### 3 Evaluation

Three experiments address the research questions: representation fidelity and validation (RQ1), a linked, policy-governed genomic workflow (RQ2), and scalability and retrieval (RQ3). Unless stated otherwise, the RQ1 and RQ3 measurements come from one set of benchmark runs of the published VCF-RDFizer v3.1.0 release, which we call the *base campaign*: 143 runs in 12 experiment families covering conversion and scaling, storage and artifacts, difficult inputs, configuration coverage, validation, and query cost (Supplementary Section S5.1). The linked workflow, the regional and large-graph retrieval runs, and two additional validation runs ran separately and are reported separately.

#### 3.1 Representation fidelity and validation

The public corpus spans five Personal Genome Project donations, an HG002 precisionFDA/Genome in a Bottle (GIAB) challenge file, the 2,504-sample 1000 Genomes chromosome-20 call set, HGSVC2 structural variants, and the HG004 and HG005 GIAB v4.2.1 benchmark VCFs [73–77]. The corpus experiment processes the first 250,000 records of eight inputs and the complete HG005 VCF, meaning all 3,856,856 of its records on chromosomes 1–22; the large cohort is excluded from the expanded-profile corpus run and assessed through controlled slices. The inputs are listed in Supplementary Table S12.

Paired validation covers the difficult-input fixtures, a 10,000-line (9,986-record) single-sample fixture under both sample profiles and three RDF artifacts, and slices up to 100,000 HG005 records. After the base campaign, a separate *consumer WGS validation run* applied the paired validation and the default shapes to the first 250,000 records of NG131FQA1I, a consumer whole-genome sequencing (WGS) VCF, producing a larger graph than any validated in the base campaign. A fault-injection experiment tests 113 deliberate corruptions with queries alone, with the default shapes, and with all shape profiles; unmutated controls test whether the validator rejects valid inputs. Separate large-graph retrieval runs compare the same thirteen answers, without SHACL, on graphs converted from one million HG005 records and from the complete HG005 VCF. The specification, exclusions, and oracle dependencies are in Supplementary Section S3.

#### 3.2 Linked genomic workflow

***\*\*Scope.\*\****

The three use cases introduced in Section 1 share a common requirement: genomic observations should remain traceable to their source while being linked to information maintained elsewhere. We therefore evaluate a single linked workflow that combines source observations with external evidence and requester-specific release rules, rather than implementing any of the three applications in full. Agreement with a conventional workflow tests whether this linked, policy-aware route reproduces the defined results. The separate frequency graph and applied rule changes then test whether external information and release conditions can be reused without modifying the

converted VCF RDF representations. Together, these experiments assess core prerequisites of the motivating applications, while clinical effectiveness, production security, and application-specific deployment remain outside the scope of this study. Supplementary Section S4 maps each motivating use case to the capabilities tested here and the additional work required for implementation.

***\*\*Application question.\*\****

The application asks: *which observed variants in the fixed 81-gene ACMG SF v3.2 secondary-findings set have a corresponding ClinVar classification, which participant and gene are associated with each match, and which matches may be released to each requester?* The benchmark fixes the gene set to ACMG SF v3.2 for reproducibility and does not represent the latest ACMG recommendations [78]. ClinVar variants are included when they have at least one review star, regardless of classification. Pathogenic and likely pathogenic (P/LP) matches are reported separately. These P/LP matches are not treated as reportable secondary findings because the workflow does not evaluate the additional gene-, inheritance-, and variant-specific criteria required for clinical reporting.

***\*\*RDF and conventional comparison workflows.\*\****

The RDF workflow links the converted VCFs to ClinVar and gene resources, materializes a release view for each requester, and queries each view with SPARQL. The conventional workflow harmonizes contig names, annotates variants with bcftools, and evaluates the same case definition in a separate Python implementation. The two workflows are compared using tuples defined by participant, gene, contig, position, and alleles. Timings are reported only when the sorted tuple sets agree exactly. Reported counts therefore represent participant-variant-gene matches rather than unique participants. Both workflows share preprocessing and the case definition, while the downstream linking, policy evaluation, and query implementations remain separate.

***\*\*Evaluation arms and simulated release policies.\*\****

We define an *experimental arm* as a distinct dataset and policy scenario used to test one aspect of the linked workflow. Four arms evaluate integration, cohort breadth, scale beyond the gene spans, and fine-grained release. Arm 1 contains the ACMG gene-span regions of five single-sample VCFs from three sources and tests integration across heterogeneous inputs. Arm 2 contains 104 unrelated high-coverage 1000 Genomes participants, four from each population, and tests the workflow across a larger cohort [79]. Arm 3 processes the complete HG005 VCF, without the gene-span restriction, to test scale. Arm 4 processes the complete NB72462M consumer WGS VCF with policies that produce different non-empty release views from the same file.

Arms 1 and 2 use normalized, split records restricted to the ACMG gene spans. For Arm 2, repeated panel-level INFO is removed from individual participant files and the required population frequencies are represented once in a separate sites-only graph. The Arm 3 result is also compared with the HG005 result from the restricted Arm 1 input. All arms use the ClinVar release from 2026-09-13 [80].

All consent and access conditions are simulated. The first three arms model requesters for clinical care, disease-specific cardiovascular research, and general research use. File-level permissions, participant withdrawals, and a restriction covering 28 cancer-predisposition genes determine the resulting release views. Arm 4 adds the participant’s physician and combines purpose-specific gene-panel, genomic-region, and single-variant rules so that each requester receives a different subset of the same VCF’s records. Each participant is represented by a single-sample VCF, so the experiment does not assess selective masking of samples within a multi-sample VCF. Four additional rule changes test where policy configuration or execution logic changes in each workflow; they do not measure usability or maintainability. Supplementary Section S4 provides the complete arm definitions, policy rules, and evaluation protocol.

### 3.3 \*\*Scalability and retrieval\*\*

We use controlled scaling *ladders* to measure how conversion costs change as dataset size increases along one dimension at a time. The *record ladder* processes progressively larger prefixes of HG005 containing 10,000, 100,000, and one million records, each with three replicates, followed by a single run on the complete HG005 VCF. The *sample ladder* holds 10,000 cohort records and their INFO content fixed while increasing the number of samples from one to 2,504 under both sample-representation profiles. For each ladder, we measure stage timings, resident memory, transient workspace, and retained artifact sizes separately. Measurements on the complete HG005 VCF and on the whole corpus are reported as single runs rather than replicated estimates.

Retrieval experiments compare individual questions, the complete thirteen-question workload, and five regional queries against both scanned and tabix-indexed VCF access. Regional queries use windows from 1 kb to 10 Mb, with twenty windows per size and three replicates. All execution routes apply the same POS-based record-selection rule so that timings are compared only after equivalent results are established. Indexing and other setup costs are reported separately from query execution.

All benchmarks ran on three dedicated hosts with the same 8-core Intel Xeon Silver 4309Y processor and 33.6 GB of RAM without swap: two identical hosts ran the base campaign and Arms 1–3 of the linked workflow, and a third ran the large-graph retrieval runs and Arm 4. Each experiment ran alone on its host inside a pinned Docker image, and timings are compared only within a host. Supplementary Section S5 lists the experiment families of the base campaign and the separate runs, with their software versions, hosts, exclusions, and archival status. All experiments and analyses can be reproduced from the public `vcf-rdfizer-testing` repository<sup>3</sup>, which contains a script for each experiment, the scripts that download and derive the inputs, the analysis and figure code, and the archived results behind the reported numbers.

---

<sup>3</sup><https://github.com/ecrum19/vcf-rdfizer-testing>

## 4 Results

### 4.1 RQ1: Source comparisons and structural validation provide complementary evidence

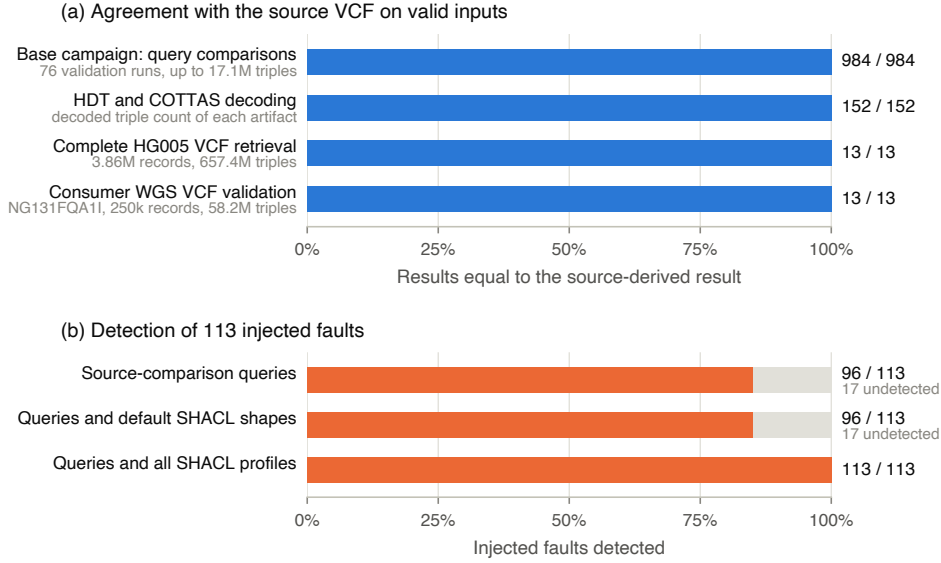
Figure 3 summarizes the observed validation outcomes against the expected result for a faithful conversion and a validator capable of detecting disagreement. Of the 78 validation runs in the base campaign, 76 completed, drawn from five experiment families: the difficult-input fixtures, the configuration covering set, the cross-encoding fixture, the operating-mode checks, and the query-cost slices of up to 100,000 HG005 records (17.1M triples). Across these runs, 984 SPARQL results had corresponding oracle results derived from the source VCF, and all 984 pairs agreed exactly after normalization. Four additional query instances were not applicable: the zero-record fixture contains no genotypes, so genotype queries Q5 and Q6 have no defined result for either of its two RDF artifacts. All 1,144 internal consistency checks also passed, including relationships between independently calculated summaries, such as the requirement that transition and transversion counts sum to the number of biallelic single-nucleotide variants.

To test whether these results depended on a particular SPARQL implementation, 25 of the 76 validation runs were repeated on four SPARQL engines. All four engines returned identical normalized answers for every query in those runs; the remaining 51 runs were executed with a single engine. Serialization fidelity was assessed separately for 152 HDT and COTTAS artifacts. After decoding, every artifact contained exactly the same number of triples as its source RDF graph.

Each validation layer tests a different property of the conversion. Query agreement establishes equality for the specified summaries, while SHACL validation establishes only the constraints enabled by the selected profile; neither demonstrates complete semantic equivalence between VCF and RDF. Decoding checks confirm triple counts after serialization, not value-by-value identity. The retrieval run on the complete HG005 VCF compared query answers without SHACL, and neither it nor the consumer WGS validation run contributes to the 984 comparisons reported for the base campaign.

In the consumer WGS validation run, the first 250,000 records of NG131FQA1I produced 58.2 million triples. All thirteen query results equaled their source-derived values, and the default shape profile, validated in record batches, reported no violations (Supplementary Section S3.2).

Of the 113 injected faults, the queries alone detected 96 (Figure 3b). The other 17, in ten fault classes, changed relationships or values that no query summary inspects, such as which allele a genotype refers to, a sample’s index, phasing, and header attribute values. The default shape profile detected none of these 17; adding the two deeper profiles detected all of them, and both unmutated control graphs still passed. On the fault-injection fixture’s graph of about 2,000 triples, the two deeper profiles took 33.7 and 73.2s, against 1.7s for the default profile. The default profile also ran in 58 base validations, on graphs of up to 0.96M triples from the 10,000-line fixture, and found no violations.



**Fig. 3 Validation outcomes on valid inputs and on injected faults.** Each bar spans the expected outcome and is filled to the observed one. (a) Results equal to those derived independently from the source VCF: the 984 applicable query comparisons of the base campaign’s 76 validation runs, the decoded triple count of each HDT and COTTAS artifact, and the thirteen queries of the separate retrieval run on the complete HG005 VCF and of the consumer WGS validation run. (b) Injected faults detected by the source-comparison queries alone, with the default SHACL shapes, and with all shape profiles.

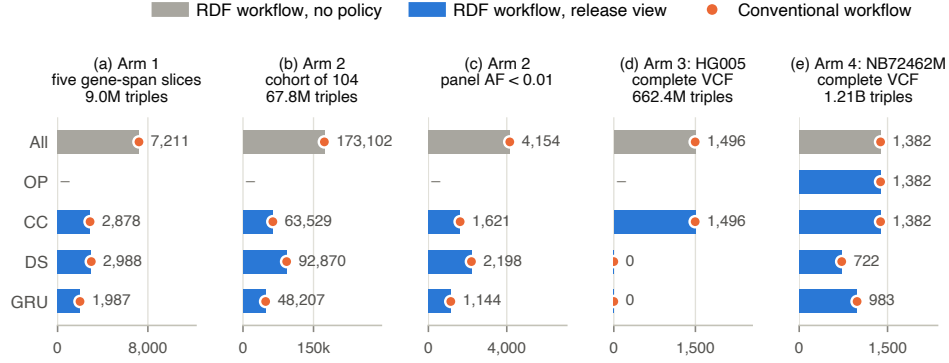
During development, this validation found three real defects, all since fixed. The predicate and class censuses (Q9 and Q10) showed that a header-only file omitted the samples its header declares; the shapes reported genotypes that referred to allele resources a raw-INFO conversion never emitted; and the policy plug-in’s own check found a withheld file’s header surviving in release views. A record count alone would have shown none of them. Version fixtures (VCFv4.1–4.5, and an undeclared version) converted, and nine of eleven difficult inputs converted and validated; a truncated gzip was refused, while a malformed sites-only fixture left that paired-coverage test incomplete. Supplementary Sections S6, S3, and S7 retain these outcomes, repeatability validation, and unresolved oracle limitations.

## 4.2 RQ2: Shared identifiers enabled reusable joins and verified, requester-specific views

Across all four arms, the RDF and bcftools workflows returned identical requester-specific match sets (Figure 4). Every materialized release view also passed the policy and source-validation checks. For HG005, the query over the complete VCF in Arm 3 reproduced the result from its gene-span slice in Arm 1 exactly, returning 1,496 participant–variant–gene matches. These counts represent match tuples rather than unique participants. The unrestricted pathogenic/likely pathogenic (P/LP) subset

contained no matches in Arms 1, 3, and 4 and eight in the Arm 2 cohort. These are ClinVar classifications retrieved under the study definition, not clinically adjudicated secondary findings.

Arm 4 extended the evaluation to fine-grained release from the complete NB72462M VCF, 5,063,417 records after normalization. The participant’s physician received every record, clinical care received all but six APOE records, general research received all but 2,520 records covered by the cancer-gene, APOE, and single-variant restrictions, and disease-specific research received only the 6,397 records within its cardiovascular panel. The RDF and conventional workflows agreed both on the released-record counts and on the resulting variant matches.



**Fig. 4 The RDF and conventional workflows returned identical match sets for every requester in every arm.** Counts are participant-variant-gene tuples, not people. Bars are the RDF workflow and dots the conventional workflow; they agree in every cell. CC denotes clinical care, DS disease-specific (cardiovascular) research, and GRU general research use; “All” applies no policy, and OP is care by the participant’s own physician, a requester only in Arm 4 (dashes elsewhere). Triple counts cover only each arm’s converted VCFs, without ClinVar or links. (c) joins the cohort’s matches to the separate panel-frequency graph at panel allele frequency (AF) < 0.01.

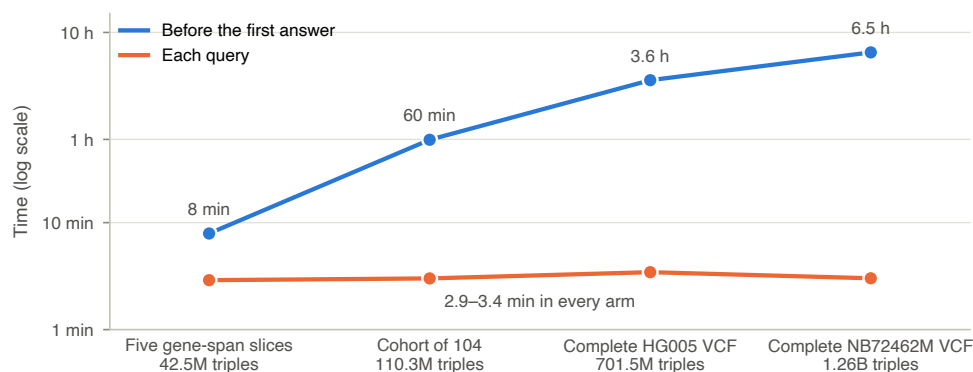
The cohort rare-variant query<sup>4</sup> illustrates the benefit of representing shared information separately. In the 1000 Genomes inputs, each participant file repeats panel-level INFO fields describing all 3,202 panel samples. These fields accounted for 93% of the RDF generated per participant, or approximately 857 million triples across the cohort. The cohort workflow removes this repeated information and instead represents the required population frequencies once in a separate graph containing 68,635 sites and 6.5 million triples. SPDI links then connect each genomic observation to ClinVar and the shared frequency graph, while the conventional workflow retrieves the same frequencies by position and allele. Both routes returned the same rare-variant matches. The reduction therefore results from storing shared information once rather than from RDF compression.

Shared variant identity also changed how one release rule could be expressed. Restricting HFE C282Y to clinical care required only a policy change in the RDF

<sup>4</sup>Query file: [cohort/rare.rq](#).

workflow, without modifying the policy engine. The conventional workflow required corresponding changes to both code and configuration data. Because the RDF rule targeted the shared variant identity, it also selected the HG002 record represented on contig 6 rather than chr6. Three other rule changes required similarly small configuration changes in both workflows. These results demonstrate reusable policy configuration for this tested case, rather than a general advantage in usability or implementation effort. Supplementary Section S4 reports the complete change-impact and linking results.

Most of the linked workflow’s computational cost occurred before query execution (Figure 5). For Arm 3, conversion of the complete HG005 VCF (3,887,810 records after normalization) and ClinVar required 54 min and linking required 8 min. Materializing, validating, and indexing the clinical requester’s view required a further 2.5 h, including 68 min for validation. Peak resource use was 13.1 GB of memory and 34 GB of disk. Once these steps were complete, individual queries required 205–209 s. Query times across all four arms ranged from 146–209 s, suggesting that the fixed ClinVar join dominated this workload rather than that SPARQL query time was independent of graph size. In Arm 3, the clinical view contained every record of the HG005 VCF while the other two views were empty. In Arm 4, materializing and validating each selective view of the 1.2-billion-triple NB72462M graph required 55–74 min.



**Fig. 5 Setup grows with the data; querying does not.** Time before the clinical-care requester’s first answer (converting and linking the arm’s files, then writing, checking, and indexing its view) and the median time of each query, for the four arms in order of graph size. The size under each arm is the number of triples QLever indexed for that arm’s unrestricted (no-policy) query: the arm’s converted VCFs, the converted ClinVar subset (33.1M triples), Arm 2’s frequency graph, and all links. These sizes are therefore larger than the VCF-only triple counts in Figure 4. Supplementary Table S9 gives every stage for every requester.

### 4.3 RQ3: Representation and workload determine the practical costs

#### 4.3.1 Sample granularity determines the amount of graph

The record ladder showed approximately linear growth in RDF output, producing 170.5–171.9 triples per VCF record. Mapping-stage resident memory remained between 1.0 and 1.7 GB as the number of records increased, while wall time and disk use increased with dataset size. These results indicate bounded memory use during mapping; they do not imply bounded memory requirements for subsequent validation, indexing, or querying.

The sample ladder revealed a substantially larger effect from the choice of sample representation (Figure 6). For 10,000 records and 2,504 samples, the expanded profile produced 627.4 million triples and a 4.10 GB HDT, compared with a 59.2 MB HDT for the condensed profile. The expanded representation contained  $432\times$  more triples, although its HDT representation was only  $69\times$  larger because compression reduced the difference in stored size. Across the sample ladder, the condensed representation’s triple count increased by only 0.9%, while HDT size increased  $5.8\times$  as the encoded sample vectors became longer. This reduction in graph size comes at the cost of direct addressability: applications using the condensed profile must decode sample vectors rather than query individual sample calls as separate RDF resources. The complete 2,504-sample chromosome-20 VCF was not converted under either profile.

#### 4.3.2 Optional artifacts should not be counted as mandatory conversion

Space-optimized storage reduced transient workspace by  $9.19\times$  on 100,000 HG005 records at a 5.2% time cost, without changing the triples. In the corpus, COTTAS occupied 0.37–0.55 times the gzip-framed N-Triples and 0.27–0.37 times HDT, but took 1.5–3.0 times longer than HDT to build. These are measured format and implementation choices, not an intrinsic requirement of semantic conversion.

Converting the complete HG005 VCF with v3.1.0 took 16.07 h including both optional builds: 20,911 s for HDT and 31,230 s for COTTAS, approximately 14.5 h together. Retaining HDT, its index, and COTTAS used a reported 10.88 GB; gzip-framed N-Triples alone occupied 2.89 GB. These are alternative artifact choices, not a single minimum storage footprint. They must not be conflated with the later, differently configured 54-minute use-case conversion (with ClinVar). Supplementary Section S8 retains per-input sizes, build times, and workspace measurements.

#### 4.3.3 Indexed retrieval benefits some questions, not every workflow

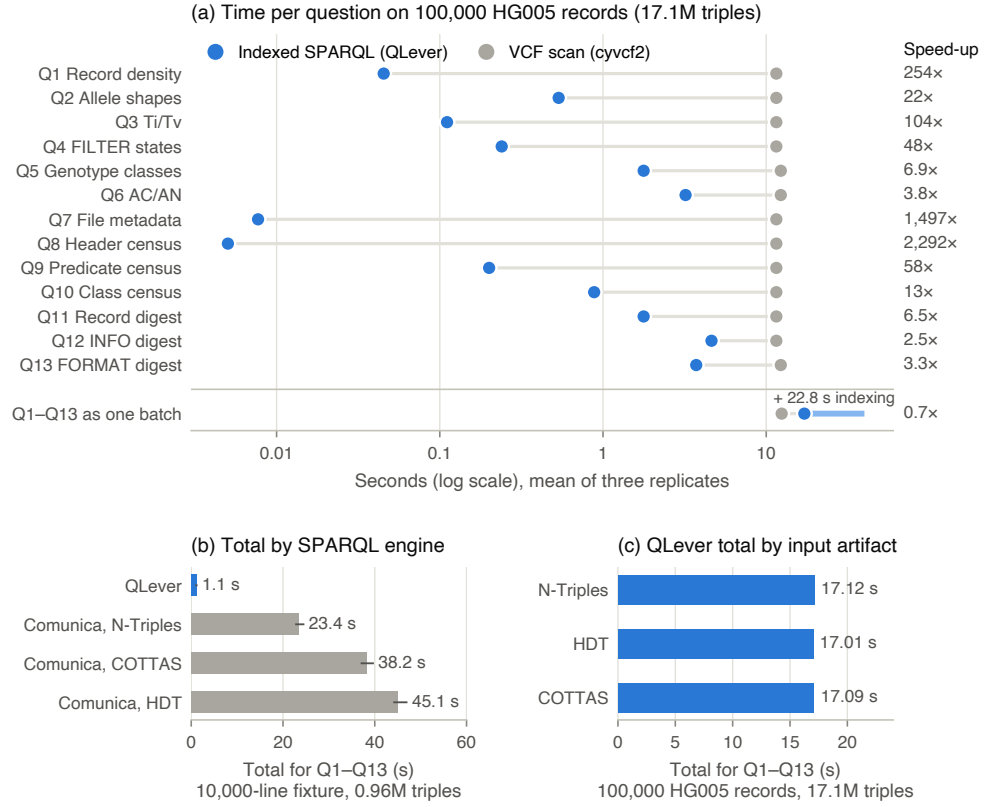
For individual questions on the 17.1M-triple graph of 100,000 HG005 records, indexed QLever took 0.005–4.62 s against cyvcf2 scans of 11.5–12.3 s (Figure 7). Yet QLever first needed a 22.8-second index, in addition to conversion. For the complete thirteen-question batch, one parser pass took 12.4 s, compared with 17.1 s of QLever queries before setup. Thus the tested single cold questions and one-pass summaries favored the parser; selected repeated questions favored the existing graph index.



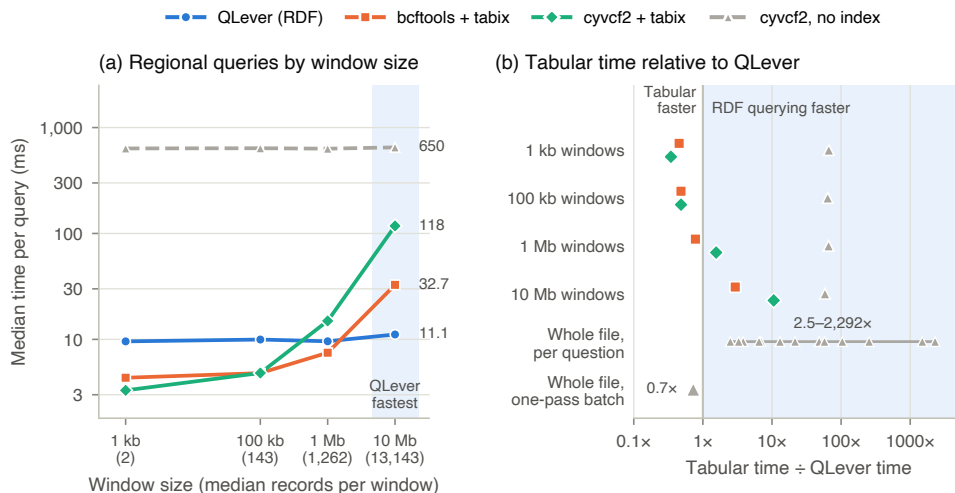
**Fig. 6 Sample representation trades addressability for size.** The same 10,000 records of the 1000 Genomes chromosome-20 VCF are converted with one to 2,504 of its samples, under each sample profile. (a) Expanded resources grow with sample calls; condensed vectors keep triple counts nearly constant. (b) Both profiles grow in stored size; the uncompressed source VCF is shown for reference. (c) At 2,504 samples the expanded-to-condensed ratio depends on the measure. The 390 $\times$  and +0.9% annotations in (a) instead compare each profile's triple count at the two ends of the ladder. Medians are shown where replicated.

Regional retrieval provides the stronger indexed-VCF baseline (Figure 8a). Across the 100,000-record HG005 graph and the 10,000-line fixture, 11,160 executions had no failures or answer disagreements. Indexed VCF was faster at 1 kb; QLever was faster for the tested 10 Mb windows. Its observed cost was nearly flat across these windows, whereas indexed-reader costs increased with the records processed. Setup remained unequal: 23.2s for QLever versus 0.48s for bgzip and tabix. Placing these regional results beside the whole-file questions (Figure 8b) shows where tabular parsing was slower than RDF querying: every unindexed regional scan, the 10-Mb windows with either indexed reader, the 1-Mb windows with cyvcf2, and each whole-file question asked on its own. It remained faster for windows up to 100 kb, for bcftools at 1 Mb, and for the one-pass batch of all thirteen questions.

On the 657.4M-triple graph of the complete HG005 VCF, QLever answered all thirteen inspection questions in 634.6s after an 868.0-second index build, with every answer equal to the parser's. This separate retrieval run compared answers without SHACL. Loading the same graph from HDT or COTTAS changed subsequent query



**Fig. 7 Retrieval cost depends on the question and execution path.** Answers agree before timings are compared. (a) Individual questions and the batch on the 17.1M-triple graph of 100,000 HG005 records, mean of three replicates; conversion is excluded and QLever indexing appears only on the batch row. Speed-up is the scan time divided by the SPARQL time. (b) Four engines on the 0.96M-triple graph of the 10,000-line fixture, excluding setup; means and ranges over nine runs. Comunica queries N-Triples and HDT directly and COTTAS through its DuckDB-backed engine. (c) QLever on the 100,000-record graph loaded from three artifacts. Similar times in (c) describe QLever's index, not native querying of those formats.



**Fig. 8 Indexed tabular readers are fastest for small regions; RDF querying is faster for 10-Mb regions and for whole-file questions asked one at a time.** Both panels use the 17.1M-triple graph of 100,000 HG005 records and the same records as a bgzip-compressed VCF; setup is excluded, and every answer agreed before timing. (a) Median time per regional query over five questions, twenty windows per size (three for the unindexed scan), and three replicates; parentheses give the median records per window. (b) The same comparisons as the ratio of tabular to QLever time, beside the thirteen whole-file questions and their one-pass batch from Figure 7a; points in the shaded region are comparisons in which tabular parsing was slower. Setup took 23.2s for QLever’s index against 0.48s for bgzip and tabix, and counting QLever’s 22.8-second index lowers the batch ratio from 0.7× to 0.3×. Supplementary Table S15 lists the medians.

time by only 0.3%, but decoding costs differed. The native HDT- and COTTAS-backed paths did not complete on the 171M-triple graph of one million HG005 records, despite answering five questions. Supplementary Table S14 reports both the completions and failures; small-fixture engine rankings do not establish large-graph feasibility.

## 5 Discussion

### 5.1 A first step toward reusable genomic information

The three motivating use cases require more than a different serialization of VCF. They require trustworthy links between observations, external evidence, clinical context, and the conditions governing use. VCF-RDFizer provides an evaluated starting point for those links: a representation that retains source context, validates selected aspects of that context, and supports a bounded integration and release workflow. It does not implement all three applications, and their potential benefits should not be read as measured outcomes of this study.

The successful bcftools comparator makes this distinction important. The integration task is not exclusive to RDF. What the semantic route adds is a common representation in which the relationships supporting the task can be stated, inspected, and reused. The separate frequency graph avoids repeating shared information in each

participant’s graph, and the identity-targeted policy expresses a new selection without changing the policy engine. These results support reuse in the tested workflow, not a general reduction in development effort or evidence that conventional pipelines cannot provide the same functions. The following applications describe where that foundation could lead and what would have to be added.

## **5.2 Clinical interpretation and reanalysis: keeping observations separate from changing evidence**

For the first use case, RDF could make the dependencies of an interpretation easier to inspect. A source observation could remain linked to the evidence assertions used at a particular time, while later assertions are added with their own provenance rather than overwriting the earlier interpretation. A query could then identify observations connected to a changed clinical assertion or frequency estimate and nominate them for review. This would address part of the reevaluation problem described in clinical guidance: recognizing when new knowledge or clinical information may affect a previous conclusion [8, 9]. It would not remove the need for clinical interpretation, nor recover a variant absent from the original call set.

Our ClinVar and frequency joins demonstrate the static relationships needed by such a workflow, not a longitudinal reanalysis service. Implementing that service would first require an explicit contract for reference assemblies, allele normalization, and unresolved matches. Minimally trimmed SPDI expressions are not necessarily NCBI’s contextual or canonical alleles in repeat regions [26]. A biologically equivalent allele assigned a different identifier can therefore miss both an evidence join and an identity-targeted policy. Normalization would need to be assessed on representative allele classes, with ambiguous or unmatched observations retained as explicit outcomes rather than silently treated as absent evidence.

The external evidence would also need to be versioned at the assertion level, retaining the submitter, disease context, review status, supporting evidence, and dates needed to distinguish a new assertion from a replacement or conflicting interpretation. Patient phenotypes, family information, and prior reports would need compatible representations and reliable links to the correct sample and observation. A change-detection process could then identify affected observations, refresh the relevant queries or indexes, and route candidate changes for clinical review. Evaluation should compare those candidates with conventional reannotation and expert-reviewed outcomes, including missed updates and unnecessary reviews. The clinical value reported for reanalysis in other studies motivates this work, but cannot be inferred from the present equality of static query results [10].

## **5.3 Pharmacogenomic decision support: connecting the genomic observation to the clinical question**

For the second use case, RDF could provide a shared representation of the path from a genomic observation to an interpreted pharmacogenomic result, a medication order, and the guidance relevant to that order. Keeping these as separate, linked resources could make it possible to inspect which observation and guideline version support

a recommendation, or to reuse an existing interpretation when a new medication is prescribed. Such a representation would address an integration concern identified in pharmacogenomic decision-support literature, rather than replace the clinical systems or interpretation tools already used [12, 13].

This application has not been implemented or evaluated here. A gene overlap or a ClinVar match does not establish a pharmacogenomic diplotype, metabolizer phenotype, or treatment recommendation. The first implementation step would be to connect the represented calls to a validated pharmacogenomic interpretation pipeline, such as a suitably configured PharmCAT workflow, and retain its assumptions, evidence, and unresolved results [2]. The input must provide the coverage, phasing, and relevant variant information required by that interpretation; the absence of a record must not automatically be treated as a reference genotype or a negative finding. Additional calling or interpretation methods would be needed wherever those requirements are not met. RDF could record these outputs and their provenance, but would not itself perform the necessary biological inference.

The next step would link the interpreted result to the correct patient, medication, indication, and other clinical context through agreed identifiers and exchange models, including FHIR Genomics where appropriate [41]. Guidelines would need versioned, computable rules and explicit handling of missing information and conflicting recommendations. The CYP2C19–clopidogrel example illustrates why this matters: a genotype-derived phenotype alone is not the complete prescribing context [11]. A clinical rule engine and EHR integration would then have to deliver the result at the appropriate point in care. Before use, the complete path would require validation with pharmacogenomic experts, assessment of recommendations against the source guidance, and testing of the clinical workflow and user interaction [12]. The present study contributes only the checked genomic representation and a demonstration of cross-resource linking on which such work could build.

#### **5.4 Consent-aware reuse: connecting explicit policies to an enforced release process**

For the third use case, RDF could make the association between a genomic resource, its governing policy, and the purpose of a request directly inspectable. Policies could refer to identified files, participants’ records, or declared selections without requiring a separate copy of the source file for every permitted use. The evaluated plug-in demonstrates part of this behavior: requester-specific views agreed with the conventional route, and ownership validation detected context that should have been withheld with its source. The views differed by participant, gene panel, purpose, region, and single variant, and arm 4 released four different, non-empty parts of one complete VCF. The study does not establish masking within one multi-sample file, and region rules match contig names as each file writes them.

An operational system would first require the governing consent and institutional conditions to be reviewed and mapped to a defined policy profile. DUO and the GA4GH Machine Readable Consent Guidance support consistent descriptions, but a computable term does not establish that a particular use is ethically or legally authorized [7, 14, 15]. The profile would need agreed meanings for purposes, permissions,

prohibitions, conflicts, and resource ownership, together with a process for resolving conditions that cannot be evaluated automatically. Default-deny and deny-overrides are choices in our implementation, not guarantees supplied by RDF or ODRL [27].

Policy evaluation must then be connected to authenticated requester credentials and an enforcement boundary that protects the source graphs, indexes, caches, and exports as well as the query endpoint [7, 28]. Policy versions, decisions, and released views would need auditable records. A withdrawal or policy change would require recomputing affected permissions and controlling future access; invalidating a local view would not by itself retract copies already released. Tests would also need to cover equivalent variant expressions, dependent resources, multi-sample representations, and information that might still be inferred from a permitted result. Security and privacy assessments are therefore separate from source-fidelity validation. The plug-in is a first step toward inspectable release decisions, not anonymization, a guarantee of regulatory compliance, or protection against access outside the evaluator. Obligations are recorded but not enforced in the evaluated implementation.

## 5.5 Validation needs to test both the graph and its oracle

The application possibilities above depend on the representation being dependable enough to reuse. The strongest validation result is therefore not simply a high pass count, but the distinction between what source comparisons detect, what structural constraints detect, and what the current implementations cannot yet assess at scale. Neither the query comparisons nor the default shapes detected all injected faults; only the costlier deeper profiles detected the complete constructed fault set. Their coverage cannot be assumed for routine large-file validation. The default profile’s clean result on 58 ordinary validations establishes conformance to those constraints, not the same fault sensitivity.

The three defects found during development demonstrate why relationships matter: declared samples, referenced alleles, and withheld-file metadata can be wrong even when record counts are correct.

Complete shape validation does not yet scale to the graph of a complete single-sample VCF. The SHACL engine holds the graph it validates in memory, at about 2.4GB per million triples on these graphs. Because each default-profile constraint concerns a single record and the file-level resources it references, that profile can be validated in record batches; on the 58.2M-triple graph of the consumer WGS validation run this took 89 minutes with four parallel workers, and at the same rate the 657.4M-triple graph of the complete HG005 VCF would take about 17 hours. The deeper profiles compare records with one another through SPARQL-based constraints, such as unique record indices, so each graph must be validated in one piece; on the 31 GB benchmark hosts the validator therefore caps them by default at 10M triples (about 24 GB), far below that graph, and their running time on large graphs has not been measured. Future work should investigate validation that does not hold the whole graph in memory, for example by evaluating cross-record constraints as queries over the index already built for retrieval. Until then, the 984 applicable comparisons of the base campaign, the separate summary agreement on the complete HG005 VCF, and the 58.2M-triple consumer WGS validation run remain distinct claims. None

establishes exhaustive semantic equivalence or the biological correctness of the original calls.

## 5.6 Representation and workload determine the deployment choice

Expanded and condensed output exchange directly addressable structure for compact vectors. That affects queries, validation, and policy targeting as well as storage. Condensed sample-level checking needs vector-aware logic, and withholding an individual value would require rewriting its vector; the implemented release demonstration does neither. Triple counts alone are consequently an inadequate measure of the cost or capability of a genotype representation.

For the tested one-pass analyses and narrow indexed lookups, native VCF access avoids setup that the RDF route cannot recover within that task. Repeated, linked, or policy-aware questions may justify a graph index, but this depends on the workload rather than following from the choice of RDF. Gzip-framed N-Triples and an appropriate index are sufficient for the demonstrated QLever path; HDT and COTTAS should be built only when their separate storage or access properties are required. The 44-question break-even point is one scenario, not a tool constant: it includes a 423.7-second conversion workflow with HDT plus 22.8 s of indexing, divided by about 10.2 s saved per separate question. It assumes the measured question mix and repeated parser scans. A batched parser pass changes the comparison, and the minimal RDF-only setup was not isolated by that estimate (Supplementary Section S8). Applications with changing evidence or policies would also need to measure the cost of updating their links, indexes, and release views; those update workloads were not benchmarked here.

## 5.7 Scope and next steps

The evidence supports a software and workflow contribution, not clinical deployment, measured usability, or superiority over other RDF converters. The converter comparison is documentation-based, and the task comparison uses bcftools. Sample scaling holds INFO fixed, the whole multi-sample cohort was not converted, and full-shape validation was limited to small inputs. The linking experiment uses single-sample files and simulated policies, and its results must remain tied to the releases and inputs that produced them.

The next steps can therefore be staged rather than treated as one general claim of clinical readiness. First, the remaining oracle coverage gaps, conformant sites-only coverage, and identity-normalization boundaries should be addressed within the converter and its validation. Second, application-specific pilots should evaluate versioned evidence updates, interpreted pharmacogenomic results linked to clinical context, or selective release under reviewed policies, with the appropriate domain experts. Finally, a deployed service would require the clinical, governance, and security assessments appropriate to its intended use. A shared-input comparison with another converter would strengthen comparative claims, while GA4GH VRS identifiers, federated queries, and multi-sample policy evaluation remain possible extensions rather

than demonstrated capabilities. VCF-RDFizer supplies a testable starting point for this progression: not the complete applications, but genomic information that can be checked, linked, and reused while their remaining requirements are made explicit.

## 6 Conclusions

VCF-RDFizer makes VCF information explicit in a representation that can be checked against its source, connected to external knowledge, and used in policy-aware research queries. The evaluation demonstrates complementary validation, reusable joins, and requester-specific result agreement with a conventional workflow, while exposing limits in oracle coverage, identity assumptions, and large-graph validation. Its measured costs support choosing semantic conversion for tasks that need those relationships and validation, rather than treating RDF as a universal replacement for native VCF processing. This is a first step toward evidence-aware reanalysis, pharmacogenomic decision support, and governed reuse; realizing those applications still requires validated interpretation, clinical integration, and enforced data-access processes.

## Availability and requirements

**Project:** VCF-RDFizer. **Home page:** <https://github.com/ecrum19/VCF-RDFizer>. **Language:** Python, with the mapping, compression, and query tools specified in Supplementary Section S5. **Platforms:** Linux, macOS, and Windows are covered by the tool’s tests; the experiments reported here ran on Ubuntu. **Distribution:** PyPI, Conda-Forge, and Docker. **License:** MIT. **Any restrictions to use by non-academics:** none. Docker supplies the pinned benchmark environment; requirements for other installations are documented with each release. v3.1.0 produced the base benchmark, v3.2.0 converted the use-case inputs, v3.3.0 releases the linking and policy extensions evaluated in pre-release builds, and v3.3.1 provided the linking for Arm 4 and the validator for the consumer WGS validation run. The release ledger distinguishes those builds rather than attributing every result to one version.

## Declarations

### Ethics approval and consent to participate

This study used publicly released genomic data and involved no recruitment of or interaction with participants. Personal Genome Project donors released their files under the project’s open-consent model [73]; GIAB and precisionFDA files are public reference materials [74, 77]; the 1000 Genomes and HGSVC2 datasets are released for research reuse [75, 76, 79]; and ClinVar is a public NCBI resource [80]. The experimental consents are simulated and do not express any real participant’s wishes. No identification of participants was attempted. Ethics approval was not sought for the reported work.

## Consent for publication

Not applicable. The manuscript reports aggregate results and software measurements, not individual-level genomic records, images, or personal details.

## Availability of data and materials

Input filenames, provider URLs, download commands, fixtures, derivation scripts, query files, manifests, normalized outputs, and comparison reports are maintained at <https://github.com/ecrum19/vcf-rdfizer-testing>. Large source VCFs are obtained from their public providers rather than redistributed in Git. Experiment 17 (`benchmarks/use_case/acmg`) contains the use-case definitions, simulated policies, conventional baseline, and applied-change tests. Its match sets, timings, and check reports, as well as the additional validation runs, are recorded under `BioMedSem_2026/benchmark-results`. No biological materials were generated or used.

The software is available at the project home page. The base benchmark used v3.1.0 (`d3b34d5`), and the linking and policy extensions are released in v3.3.0 (`7ab2300`; image `sha256:1838720d...06432`). The use case ran pre-release builds, including `578c2be` for Arm 3, rather than the exact final v3.3.0 release. Its tested inputs did not meet the empty-selector or no-VCF-triples conditions subsequently refused by the plug-ins. Supplementary Section [S5](#) identifies the pre-release and later-run commits, the v3.1.0 image digest, and the bundled shape snapshot. The public vocabulary is separately versioned [\[51\]](#). Results must be traced to their recorded build and input, not only to the latest package.

**[AUTHOR QUERY: Add a persistent archive identifier containing the exact evaluated code, dirty-tree changes for use-case arms 1–2, complete image digests, vocabulary/shapes, queries, and evidence manifests. Include the commits and configurations of the default-shape mutation rerun and the consumer WGS validation runs recorded in the release ledger. The supplied manuscript package does not itself establish this complete archived snapshot.]**

## Competing interests

None declared.

## Funding

Elias Crum acknowledges support from the Research Foundation–Flanders (FWO), SB Fellowship 1S27825N.

**[AUTHOR QUERY: Complete the funding statements for B.B., R.T., and G.E.]**

## Authors' contributions

E.C., G.E., and R.T. conceived of this study. E.C. did the implementation and coding work. E.C. devised the validation and use-case demonstration with the guidance of

R.T. and executed the experiments. E.C., G.E., and R.T. reviewed and verified all results. E.C., B.B., G.E., and R.T. wrote and edited the manuscript.

## **Acknowledgments**

Benchmarking used the resources of IDLab at Ghent University. Claude Opus 5.5 [81] and GPT-6 Astra [82] assisted with experiment automation scripting, result data analysis scripting, and restructuring and language editing of this manuscript. The authors reviewed all AI outputs and take full responsibility for the manuscript, experiments, tools, and repositories.

## References

- [1] SAM/BAM Format Specification Working Group: The Variant Call Format Specification (2024). <https://github.com/samtools/hts-specs>
- [2] Li, B., Sangkuhl, K., Keat, K., Whaley, R.M., Woon, M., Verma, S., Dudek, S., Tuteja, S., Verma, A., Whirl-Carrillo, M., Ritchie, M.D., Klein, T.E.: How to run the pharmacogenomics clinical annotation tool (pharmcat). *Clinical Pharmacology & Therapeutics* **113**(5), 1036–1047 (2023) <https://doi.org/10.1002/cpt.2790>
- [3] Souche, E., Beltran, S., Brosens, E., Belmont, J.W., Fossum, M., Riess, O., Gilissen, C., Ardeshirdavani, A., Houge, G., Van Gijn, M., Clayton-Smith, J., Synofzik, M., De Leeuw, N., Deans, Z.C., Dincer, Y., Eck, S.H., Van Der Crabben, S., Balasubramanian, M., Graessner, H., Sturm, M., Firth, H., Ferlini, A., Nab-bout, R., De Baere, E., Liehr, T., Macek, M., Matthijs, G., Scheffer, H., Bauer, P., Yntema, H.G., Weiss, M.M.: Recommendations for whole genome sequencing in diagnostics for rare diseases. *European Journal of Human Genetics* **30**(9), 1017–1021 (2022) <https://doi.org/10.1038/s41431-022-01113-x>
- [4] Danecek, P., Bonfield, J.K., Liddle, J., Marshall, J., Ohan, V., Pollard, M.O., Whitwham, A., Keane, T., McCarthy, S.A., Davies, R.M., Li, H.: Twelve years of samtools and bcftools. *GigaScience* **10**(2), 008 (2021) <https://doi.org/10.1093/gigascience/giab008>
- [5] Gottesman, O., Kuivaniemi, H., Tromp, G., Faucett, W.A., Li, R., Manolio, T.A., *et al.*: The Electronic Medical Records and Genomics (eMERGE) Network: past, present, and future. *Genetics in Medicine* **15**(10), 761–771 (2013) <https://doi.org/10.1038/gim.2013.72>
- [6] The All of Us Research Program Genomics Investigators: Genomic data in the All of Us research program. *Nature* **627**(8003), 340–346 (2024) <https://doi.org/10.1038/s41586-023-06957-x>
- [7] Rehm, H.L., *et al.*: GA4GH: International policies and standards for data sharing across genomic research and healthcare. *Cell Genomics* **1**(2), 100029 (2021) <https://doi.org/10.1016/j.xgen.2021.100029>
- [8] Richards, S., Aziz, N., Bale, S., Bick, D., Das, S., *et al.*: Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine* **17**(5), 405–424 (2015) <https://doi.org/10.1038/gim.2015.30>
- [9] Deignan, J.L., Chung, W.K., Kearney, H.M., Monaghan, K.G., Rehder, C.W., Chao, E.C.: Points to consider in the reevaluation and reanalysis of genomic test results: a statement of the American College of Medical Genetics and Genomics

- (ACMG). *Genetics in Medicine* **21**(6), 1267–1270 (2019) <https://doi.org/10.1038/s41436-019-0478-1>
- [10] Hiatt, S.M., Amaral, M.D., Bowling, K.M., Finnila, C.R., Thompson, M.L., Gray, D.E., Lawlor, J.M.J., Cochran, J.N., Bebin, E.M., Brothers, K.B., East, K.M., Kelley, W.V., Lamb, N.E., Levy, S.E., Lose, E.J., Neu, M.B., Rich, C.A., Simmons, S., Myers, R.M., Barsh, G.S., Cooper, G.M.: Systematic reanalysis of genomic data improves quality of variant interpretation. *Clinical Genetics* **94**(1), 174–178 (2018) <https://doi.org/10.1111/cge.13259>
  - [11] Lee, C.R., Luzum, J.A., Sangkuhl, K., Gammal, R.S., Sabatine, M.S., Stein, C.M., Kisor, D.F., Limdi, N.A., Lee, Y.M., Scott, S.A., Hulot, J.-S., Roden, D.M., Gaedigk, A., Caudle, K.E., Klein, T.E., Johnson, J.A., Shuldiner, A.R.: Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update. *Clinical Pharmacology & Therapeutics* **112**(5), 959–967 (2022) <https://doi.org/10.1002/cpt.2526>
  - [12] Wake, D.T., Smith, D.M., Kazi, S., Dunnenberger, H.M.: Pharmacogenomic clinical decision support: A review, how-to guide, and future vision. *Clinical Pharmacology & Therapeutics* **112**(1), 44–57 (2022) <https://doi.org/10.1002/cpt.2387>
  - [13] Johnson, D., Del Fiol, G., Kawamoto, K., Romagnoli, K.M., Sanders, N., Isaacson, G., Jenkins, E., Williams, M.S.: Genetically guided precision medicine clinical decision support tools: a systematic review. *Journal of the American Medical Informatics Association* **31**(5), 1183–1194 (2024) <https://doi.org/10.1093/jamia/ocae033>
  - [14] Lawson, J., Cabili, M.N., Kerry, G., *et al.*: The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **1**(2), 100028 (2021) <https://doi.org/10.1016/j.xgen.2021.100028>
  - [15] Global Alliance for Genomics and Health: Machine Readable Consent Guidance (MRCG). Technical implementation guidance, version 1.0. Accessed 7 October 2026. <https://www.ga4gh.org/product/machine-readable-consent-guidance/> Accessed 2026-10-07
  - [16] European Parliament and Council of the European Union: Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 4 May 2016, pp. 1–88 (2016). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
  - [17] Shabani, M., Borry, P.: Rules for processing genetic data for research purposes in view of the new EU General Data Protection Regulation. *European Journal of Human Genetics* **26**(2), 149–156 (2018) <https://doi.org/10.1038/>

- [18] Paila, U., Chapman, B.A., Kirchner, R., Quinlan, A.R.: GEMINI: integrative exploration of genetic variation and genome annotations. *PLoS Computational Biology* **9**(7), 1003153 (2013) <https://doi.org/10.1371/journal.pcbi.1003153>
- [19] Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1: Concepts and Abstract Syntax (2014). <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>
- [20] Hogan, A., Blomqvist, E., Cochez, M., D’Amato, C., Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A.: Knowledge graphs. *ACM Computing Surveys* **54**(4) (2021) <https://doi.org/10.1145/3447772>
- [21] World Wide Web Consortium (W3C): LinkedData. <https://www.w3.org/wiki/LinkedData> Accessed 2026-03-04
- [22] Belleau, F., Nolin, M.-A., Tourigny, N., Rigault, P., Morissette, J.: Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics* **41**(5), 706–716 (2008) <https://doi.org/10.1016/j.jbi.2008.03.004>
- [23] Breit, A., Ott, S., Agibetov, A., Samwald, M.: OpenBioLink: A benchmarking framework for large-scale biomedical link prediction (2019) <https://doi.org/10.48550/ARXIV.1912.04616> . Publisher: arXiv Version Number: 2
- [24] SIB Swiss Institute of Bioinformatics RDF Group Members, Altenhoff, A., Bairoch, A., Bansal, P., Baratin, D., Bastian, F., Bolleman\*, J., Bridge, A., Burdet, F., Cramer, K., Dauvillier, J., Dessimoz, C., Gehant, S., Glover, N., Gnodtke, K., Hayes, C., Ibberson, M., Kriventseva, E., Kuznetsov, D., Frédérique, L., Mehl, F., Mendes De Farias\*, T., Michel, P.-A., Moretti, S., Morgat, A., Österle, S., Pagni, M., Redaschi, N., Robinson-Rechavi, M., Samarasinghe, K., Sima, A.-C., Szklarczyk, D., Topalov, O., Touré, V., Unni, D., Von Mering, C., Wollbrecht, J., Zahn-Zabal\*, M., Zdobnov, E.: The SIB Swiss Institute of Bioinformatics Semantic Web of data. *Nucleic Acids Research* **52**(D1), 44–51 (2024) <https://doi.org/10.1093/nar/gkad902>
- [25] Harris, S., Seaborne, A., Prud’hommeaux, E.: SPARQL 1.1 Query Language (2013). <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>
- [26] Holmes, J.B., Moyer, E., Phan, L., Maglott, D., Kattman, B.: SPDI: data model for variants and applications at NCBI. *Bioinformatics* **36**(6), 1902–1907 (2020) <https://doi.org/10.1093/bioinformatics/btz856>
- [27] Iannella, R., Villata, S.: ODRL Information Model 2.2. W3C (2018). <https://www.w3.org/TR/odrl-model/> Accessed 2025-11-27

- [28] OASIS: eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard (2013). <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>
- [29] Katayama, T., Kawashima, S., Micklem, G., Kawano, S., Kim, J.-D., Kocbek, S., Okamoto, S., Wang, Y., Wu, H., Yamaguchi, A., *et al.*: Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research* **8**, 1677 (2019) <https://doi.org/10.12688/f1000research.18238.1>
- [30] Lindenbaum, P.: JVarkit: java-based utilities for Bioinformatics. figshare (2015). <https://doi.org/10.6084/m9.figshare.1425030.v1> . [https://figshare.com/articles/journal\\_contribution/JVarkit\\_java\\_based\\_utilities\\_for\\_Bioinformatics/1425030](https://figshare.com/articles/journal_contribution/JVarkit_java_based_utilities_for_Bioinformatics/1425030)
- [31] Garrison, E., Kronenberg, Z.N., Dawson, E.T., Pedersen, B.S., Prins, P.: A spectrum of free software tools for processing the VCF variant call format: vcfliib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology* **18**(5), 1009123 (2022) <https://doi.org/10.1371/journal.pcbi.1009123>
- [32] Mitsuhashi, N., Toyo-oka, L., Katayama, T., Kawashima, M., Kawashima, S., Miyazaki, K., Takagi, T.: TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation* **9**(1), 44 (2022) <https://doi.org/10.1038/s41439-022-00222-9>
- [33] Bodrug-Schepers, A., Chabane, H., Montoya, G., Serrano-Alvarado, P., Redon, R., Gaignard, A.: Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In: *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*. CEUR Workshop Proceedings, vol. 4196. CEUR Workshop Proceedings, Barcelona, Spain (2025). [https://ceur-ws.org/Vol-4196/paper\\_16.pdf](https://ceur-ws.org/Vol-4196/paper_16.pdf) Accessed 2025-11-24
- [34] Böhmer, J.K., Janssen, R.: Linked data in practice: An RDF-based approach with SPARQLing Genomics. Zenodo. Zenodo record, version 4 (2020). <https://doi.org/10.5281/zenodo.3903111>
- [35] Janssen, R.: SPARQLing Genomics. Open-source software. Accessed 2026-09-03 (2019). <https://gitlab.com/roelj/sparqling-genomics>
- [36] Baran, J., Durgahee, B.S.B., Eilbeck, K., Antezana, E., Hoehndorf, R., Dumontier, M.: GFVO: the Genomic Feature and Variation Ontology. *PeerJ* **3**, 933 (2015) <https://doi.org/10.7717/peerj.933>
- [37] Hail Team: Hail: scalable genomic data analysis. Accessed October 2026 (2026). <https://github.com/hail-is/hail>
- [38] TileDB, Inc.: TileDB-VCF: efficient variant-call data storage and retrieval.

Accessed October 2026 (2026). <https://github.com/TileDB-Inc/TileDB-VCF>

- [39] Rambla, J., Baudis, M., Ariosa, R., *et al.*: Beacon v2 and Beacon networks: A “lingua franca” for federated data discovery in biomedical genomics, and beyond. *Human Mutation* **43**(6), 791–799 (2022) <https://doi.org/10.1002/humu.24369>
- [40] Kelleher, J., Lin, M., Albach, C.H., Birney, E., *et al.*: htsget: a protocol for securely streaming genomic data. *Bioinformatics* **35**(1), 119–121 (2019) <https://doi.org/10.1093/bioinformatics/bty492>
- [41] HL7 International: HL7 FHIR Genomics Reporting Implementation Guide. Accessed October 2026 (2026). <https://hl7.org/fhir/uv/genomics-reporting/>
- [42] Penha, E.D.S., Iriabho, E., Dussaq, A., Oliveira, D.M., Almeida, J.S.: Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics* **33**(4), 547–548 (2017) <https://doi.org/10.1093/bioinformatics/btw652>
- [43] Strozzi, F.: bioruby-vcf2rdf: simple converter from VCF format to RDF Turtle format. GitHub repository, BioHackathon 2013. Accessed 2026-10-07 (2013). <https://github.com/fstrozzi/bioruby-vcf2rdf>
- [44] BioInterchange: BioInterchange 2: genomic linked data converter for GFF, GVF, and VCF via JSON-LD. GitHub repository, version 2.0.5. Accessed 2026-10-07 (2021). <https://github.com/BioInterchange/BioInterchangeC>
- [45] Bendiken, A., Bonnal, R.J.P., Strozzi, F.: ruby-rdf/rdf-vcf: RDF VCF parser (2015). <https://github.com/ruby-rdf/rdf-vcf> Accessed 2025-03-01
- [46] Prasanna, S., Kumar, A., Rao, D., Simoes, E.J., Rao, P.: A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data* **7**, 1466391 (2025) <https://doi.org/10.3389/fdata.2024.1466391>
- [47] Larmande, P., Pittolat, B., Tando, N., Pomie, Y., Happi Happi, B.G., Guignon, V., Ruiz, M.: AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data* **26**(Suppl 1), 73 (2025) <https://doi.org/10.1186/s12863-025-01359-6>
- [48] Touré, V., Unni, D., Krauss, P., Abdelwahed, A., Buchhorn, J., Hinderling, L., Geiger, T.R., Österle, S.: The SPHN Schema Forge – transform healthcare semantics from human-readable to machine-readable by leveraging semantic web technologies. *Journal of Biomedical Semantics* **16**, 9 (2025) <https://doi.org/10.1186/s13326-025-00330-9>
- [49] Gaignard, A., Rosnet, T., Lamotte, F., Lefort, V., Devignes, M.-D.: FAIR-Checker: supporting digital resource findability and reuse with knowledge graphs and semantic web standards. *Journal of Biomedical Semantics* **14**, 7 (2023) <https://doi.org/10.1186/s13326-023-00289-5>

- [50] Ansari, A., Conte, M., Flynn, A., Paturkar, A.: A prototype ETL pipeline that uses HL7 FHIR RDF resources when deploying pure functions to enrich knowledge graph patient data. *Journal of Biomedical Semantics* **16**, 16 (2025) <https://doi.org/10.1186/s13326-025-00335-4>
- [51] Crum, E.: VCF Core Vocabulary. Version 2.1.3. GitHub repository. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026 (2026). <https://github.com/ecrum19/vcf-core-vocabulary/tree/8a5f23a>
- [52] Crum, E., Buelens, B., Ertaylan, G., Taelman, R.: VCF Core Vocabulary: A Semantic Foundation for RDF Representations of VCF. Manuscript in preparation for SWAT4HCLS 2027; preprint at commit 9507dd9 (2026). [https://github.com/ecrum19/SWAT4HCLS\\_2027/blob/9507dd9/long\\_revised/main\\_long.pdf](https://github.com/ecrum19/SWAT4HCLS_2027/blob/9507dd9/long_revised/main_long.pdf)
- [53] Landrum, M.J., Lee, J.M., Benson, M., Brown, G.R., Chao, C., Chitipiralla, S., Gu, B., Hart, J., Hoffman, D., Jang, W., Karapetyan, K., Katz, K., Liu, C., Maddipatla, Z., Malheiro, A., McDaniel, K., Ovetsky, M., Riley, G., Zhou, G., Holmes, J.B., Kattman, B.L., Maglott, D.R.: ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research* **46**(D1), 1062–1067 (2018) <https://doi.org/10.1093/nar/gkx1153>
- [54] VCF-RDFizer maintainers: vcf-rdfizer. Python Package Index (PyPI), version 3.3.0. Accessed 2026-10-07 (2026). <https://pypi.org/project/vcf-rdfizer/>
- [55] VCF-RDFizer maintainers: vcf-rdfizer. conda-forge, version 3.3.0. Accessed 2026-10-07 (2026). <https://anaconda.org/conda-forge/vcf-rdfizer>
- [56] VCF-RDFizer maintainers: ecrum19/vcf-rdfizer. Docker Hub image, tag 3.3.0. Accessed 2026-10-07 (2026). <https://hub.docker.com/r/ecrum19/vcf-rdfizer>
- [57] Bolleman, J.T., Mungall, C.J., Strozzi, F., Baran, J., Dumontier, M., Bonnal, R.J.P., Buels, R., Hoehndorf, R., Fujisawa, T., Katayama, T., Cock, P.J.A.: FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics* **7**(1), 39 (2016) <https://doi.org/10.1186/s13326-016-0067-z>
- [58] Eilbeck, K., Lewis, S.E., Mungall, C.J., Yandell, M., Stein, L., Durbin, R., Ashburner, M.: The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology* **6**(5), 44 (2005) <https://doi.org/10.1186/gb-2005-6-5-r44>
- [59] Menotti, L., Cazzaro, M., Rueda, M., Gut, I.G., Silvello, G.: HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In: *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*. CEUR Workshop Proceedings, Barcelona, Spain (2025). [https://www.dei.unipd.it/~silvello/papers/2025\\_MCS\\_SWAT4HCLS.pdf](https://www.dei.unipd.it/~silvello/papers/2025_MCS_SWAT4HCLS.pdf)

- [60] Wagner, A.H., Babb, L., Alterovitz, G., Baudis, M., Brush, M., Cameron, D.L., Cline, M., Griffith, M., Griffith, O.L., Hunt, S.E., Kreda, D., Lee, J.M., Li, S., Lopez, J., Moyer, E., Nelson, T., Patel, R.Y., Riehle, K., Robinson, P.N., Rynearson, S., Schuilenburg, H., Tsukanov, K., Walsh, B., Konopko, M., Rehm, H.L., Yates, A.D., Freimuth, R.R., Hart, R.K.: The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. *Cell Genomics* **1**(2), 100027 (2021) <https://doi.org/10.1016/j.xgen.2021.100027>
- [61] Wagner, A.H., Hart, R., Babb, L., Kuzma, K., Schuilenburg, H., Stevenson, J., O'Neill, T., Epic, M.-T., Holmes, B., Moyer, E., Baudis, M., Robinson, P., Cline, M., Patel, R.Y., Freimuth, R.R.: ga4gh/vrs 2.0.1. Zenodo (2025). <https://doi.org/10.5281/zenodo.15059674> . <https://doi.org/10.5281/zenodo.15059674> Accessed 2026-03-02
- [62] De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML) (2024). <https://rml.io/specs/rml/> Accessed 2025-11-24
- [63] Oo, S.M., Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In: Sattler, U., Hogan, A., Keet, M., Presutti, V., Almeida, J.P.A., Takeda, H., Monnin, P., Pirrò, G., d'Amato, C. (eds.) *The Semantic Web – ISWC 2022* vol. 13489, pp. 697–713. Springer, Cham (2022). [https://doi.org/10.1007/978-3-031-19433-7\\_40](https://doi.org/10.1007/978-3-031-19433-7_40) . Series Title: *Lecture Notes in Computer Science*. [https://link.springer.com/10.1007/978-3-031-19433-7\\_40](https://link.springer.com/10.1007/978-3-031-19433-7_40) Accessed 2025-11-24
- [64] Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web* **19**, 22–41 (2013)
- [65] Arenas-Guerrero, J., Ferrada, S.: COTTAS: Columnar triple table storage for efficient and compressed RDF management. In: *The Semantic Web – ISWC 2025. Lecture Notes in Computer Science*, vol. 16141, pp. 313–331. Springer, Cham (2026). [https://doi.org/10.1007/978-3-032-09530-5\\_18](https://doi.org/10.1007/978-3-032-09530-5_18) . [https://doi.org/10.1007/978-3-032-09530-5\\_18](https://doi.org/10.1007/978-3-032-09530-5_18)
- [66] Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A General-Purpose Data Compressor. *ACM Transactions on Information Systems* **37**(1), 1–30 (2019) <https://doi.org/10.1145/3231935>
- [67] Bast, H., Buchhold, B.: QLever: A query engine for efficient SPARQL+Text search. In: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pp. 647–656 (2017). <https://doi.org/10.1145/3132847.3132921> . <https://doi.org/10.1145/3132847.3132921>
- [68] Taelman, R., Herwegen, J.V., Sande, M.V., Verborgh, R.: Comunica: A modular

- SPARQL query engine for the web. In: Proceedings of the 17th International Semantic Web Conference, pp. 239–255. Springer, Cham (2018). [https://doi.org/10.1007/978-3-030-00668-6\\_15](https://doi.org/10.1007/978-3-030-00668-6_15) . <https://comunica.dev/>
- [69] Crum, E.: Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas` (2026). <https://github.com/ecrum19/comunica-feature-cottas>
  - [70] Danecek, P., Auton, A., Abecasis, G., Albers, C.A., Banks, E., DePristo, M.A., Handsaker, R.E., Lunter, G., Marth, G.T., Sherry, S.T., McVean, G., Durbin, R., 1000 Genomes Project Analysis Group: The variant call format and VCFtools. *Bioinformatics* **27**(15), 2156–2158 (2011) <https://doi.org/10.1093/bioinformatics/btr330>
  - [71] Pedersen, B.S., Quinlan, A.R.: cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics* **33**(12), 1867–1869 (2017) <https://doi.org/10.1093/bioinformatics/btx057>
  - [72] World Wide Web Consortium: Shapes Constraint Language (SHACL). W3C Recommendation (2017). <https://www.w3.org/TR/shacl/> Accessed 2026-10-02
  - [73] Personal Genome Project (2005). [https://my.pgp-hms.org/public\\_genetic\\_data](https://my.pgp-hms.org/public_genetic_data) Accessed 2025-11-24
  - [74] NIST Genome in a Bottle (2025). <https://www.nist.gov/programs-projects/genome-bottle> Accessed 2025-11-24
  - [75] The 1000 Genomes Project Consortium: A global reference for human genetic variation. *Nature* **526**(7571), 68–74 (2015) <https://doi.org/10.1038/nature15393>
  - [76] Ebert, P., Audano, P.A., Zhu, Q., Rodriguez-Martin, B., Porubsky, D., Bonder, M.J., Sulovari, A., Ebler, J., Zhou, W., Serra Mari, R., *et al.*: Haplotype-resolved diverse human genomes and integrated analysis of structural variation. *Science* **372**(6537), 7117 (2021) <https://doi.org/10.1126/science.abf7117>
  - [77] Wagner, J., Olson, N.D., Harris, L., Khan, Z., Farek, J., Mahmoud, M., Stankovic, A., Kovacevic, V., Yoo, B., Miller, N., *et al.*: Benchmarking challenging small variants with linked and long reads. *Cell Genomics* **2**(5), 100128 (2022) <https://doi.org/10.1016/j.xgen.2022.100128>
  - [78] Miller, D.T., Lee, K., Abul-Husn, N.S., Amendola, L.M., *et al.*: ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). *Genetics in Medicine* **25**(8), 100866 (2023) <https://doi.org/10.1016/j.gim.2023.100866>

- [79] Byrska-Bishop, M., Evani, U.S., Zhao, X., *et al.*: High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell* **185**(18), 3426–344019 (2022) <https://doi.org/10.1016/j.cell.2022.08.004>
- [80] National Center for Biotechnology Information: ClinVar VCF Release (GRCh38). Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). [https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf\\_GRCh38/](https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/)
- [81] Anthropic: Claude Opus 5.5 (Extra) [Large language model]. Accessed through the Claude Code desktop application. Used September 2026 (2026). <https://www.anthropic.com/claude>
- [82] OpenAI: GPT-6 Astra (X-High) [Large language model]. Accessed through the Codex desktop application. Used September 2026 (2026). <https://openai.com/>